

Addendum A — Runtime Trace and Training-Log Findings

Extends *SafeTail 2.0 — Codebase Analysis & Research-Paper Traceability Report* (§1–§10)

Prepared for Krishna Shukla | Project: IP_Arani | 19 August 2026 | Sections §11–§13

11. Scope, and one correction to the base report

Sections §1–§10 audited the source against the three specification documents. This addendum adds what that audit did not cover: findings from tracing the *running* system end to end (request creation → arrival → dispatch → server admission → reward → replay), and from analysing the *shipped training logs* in `results/safetail_training_logs/`. Section §11.1 corrects one reading in the base report's vicinity. §12 is new findings. §13 revises the remediation priorities in §10.

The most consequential item is **§12.4**: the headline baseline comparison in BTP Figure 5.2 is not budget-controlled, and the effect is large enough to account for an unknown share of the reported tail-latency win.

11.1 Correction — the agent's state is not $(\beta+1)$ -dimensional CORRECTION

BTP §4.2 states the state is a $(\beta+1)$ -dimensional vector of per-server load fractions, and `constants.py:27` appears to confirm it with `nS = 1 * beta + 1`. Both are misleading. `nS` is passed into the agent — `DQNAgent(states=constants.nS, ...)` at `controller.py:60`, stored as `self.nS` at `agent.py:123` — and then **never used again**. It is the only occurrence of the attribute in the file. The network's input layer is declared `keras.Input(shape=(None, 1))` (`agent.py:231`), so the constant cannot and does not constrain the state dimension.

The state actually passed to the network is built by `agent.request_to_state_array` (`agent.py:37`), which iterates `vars(request_obj)` and flattens every numeric attribute of the `Request`. Dictionaries are skipped, so `server_dicts` is excluded — but `server_np` is a *list* of arrays, so it takes the ragged branch and is included in full: per-server RAM, GPU usage and memory, execution and processing times, cores used, capacities, per-core CPU usage, and CPU/GPU clocks (`user.py:159–227`).

Claim	Reality in code	Consequence
BTP §4.2: state is $\beta+1$ load fractions	State is the full flattened <code>Request</code> , variable length, including <code>server_np</code>	The report understates its own system, and contradicts its §3.2
<code>constants.nS = 6</code>	Passed to the agent and stored, then never used; input layer is <code>(None, 1)</code>	Misleads any reader auditing the state dimension
Contribution #1: encoder for variable-length heterogeneous state	Justified. The input genuinely is variable-length	The encoder + GlobalAveragePooling is doing real work, not solving a non-problem

Net effect: the encoder contribution stands. What fails is the report's own description of it. Anyone reading BTP §4.2 alone would conclude the architecture is unnecessary.

12. New findings

12.1 The step reward has no redundancy cost, and is monotone increasing in $|A|$ HIGH

`compute_step_reward` (`controller.py:285`) allocates `rewards = np.zeros(num_servers)` and fills only the indices in `action_subset` , leaving unselected servers at `0.0` . Each filled entry is `log(product + 1.0)` (`controller.py:434`), bounded in `[0, log 2]` . The controller then collapses it with `np.mean(final_step_reward_list)` (`controller.py:676`) — a mean over *all five slots*, divided by a constant.

The step reward is therefore $(1/5) \cdot \sum_{i \in A} \log(1 + \text{headroom}_i)$. Every additional server contributes a non-negative term to a fixed denominator, so **adding any server weakly increases the reward, always**. No term anywhere in either reward level is decreasing in $|A|$. The compute cost of redundancy is unpriced.

The episodic reward does not compensate: its three terms are the discounted step sum (increasing in $|A|$), the satisfaction ω (increasing, since completion is a min over the subset), and a mean-waiting-time penalty that redundancy also reduces. All three push the same direction.

12.2 The agent learns to over-replicate HIGH

The prediction from §12.1 is borne out. Recomputing the true per-episode selection rate from `request_wise_access_log.csv` (15,225 requests, 15 per episode):

Statistic	Value
Per-episode rate, first 50 episodes	0.51 (K $\approx$ 2.5 of 5)
Per-episode rate, final 50 episodes	0.78 (K $\approx$ 3.9 of 5)
Mean over the whole run	0.711 (K $\approx$ 3.55)
Request-wise median	0.8 (4 of 5 servers)
Requests dispatched to exactly 4 servers	11,253 of 15,225 (73.9%)
Episodes whose rate exceeds the K=3 baseline ceiling	83.3%

The trajectory is a monotone climb, not an adaptation. This directly contradicts BTP §6.2, which reads the same quantity as the model "adapt[ing] the number of candidate servers per request, using greater redundancy when there is flexibility."

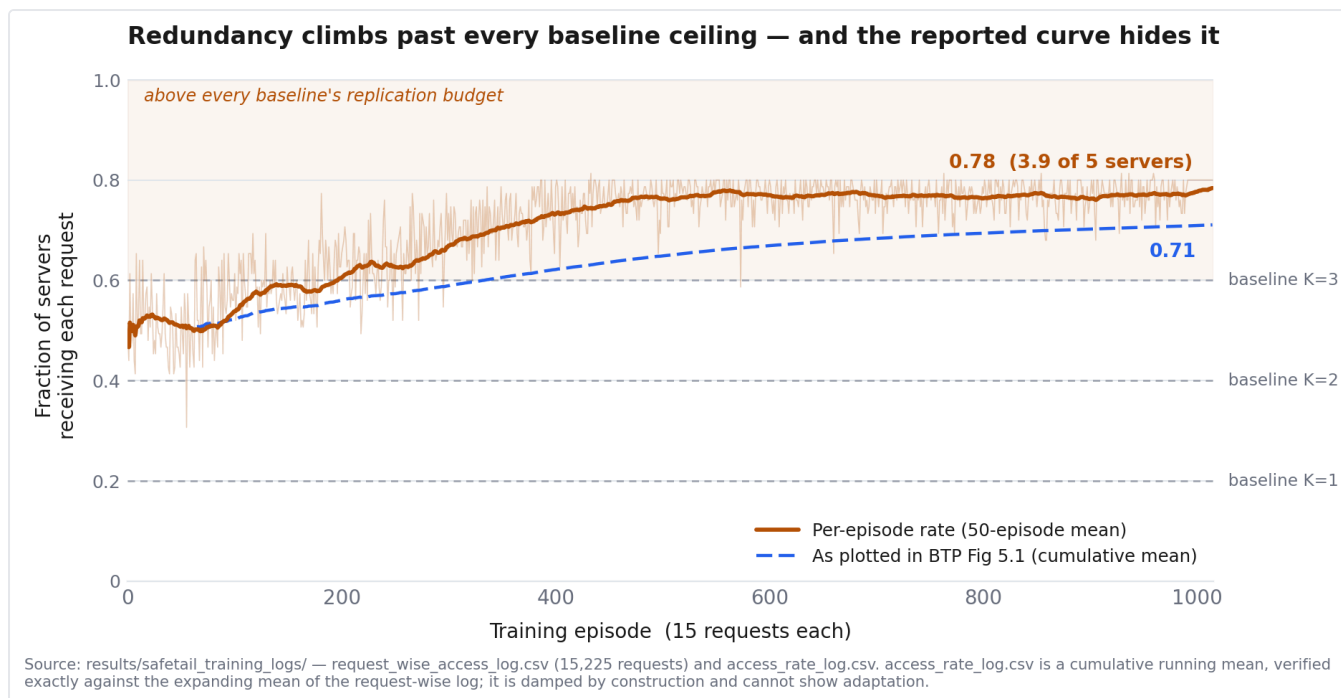


Figure A.1. True per-episode replication rate (orange) against the cumulative curve reported in BTP Fig 5.1 (blue dashed). Dashed grey lines mark the fixed replication budgets of the `minload_K` / `minprop_K` / `rand_K` baselines. The 50-episode mean crosses the $K=3$ ceiling at episode 197 and never returns below it; 83.3% of individual episodes sit above that ceiling.

12.3 The reported access-rate curve is a cumulative mean MEDIUM

`access_rate_log.csv` is not a per-episode series. Its value at episode i equals the expanding mean of the first $15(i+1)$ entries of `request_wise_access_log.csv` — verified exactly at episodes 0, 10, 100, 500 and 1014, and its final value (0.7105812807881775) is bit-identical to the overall request-wise mean.

A cumulative average is damped by construction: it cannot exhibit non-monotonic behaviour, and it understates the endpoint (0.71 reported versus 0.78 actual). The bottom row of BTP Fig 5.1 is therefore the wrong statistic for the claim §6.2 draws from it, in addition to that claim being wrong on the merits.

12.4 The baseline comparison is confounded by replication budget HIGH

Every baseline is capped at a fixed $K \in \{1, 2, 3\}$ servers (`controller.py:622–639` , via `_select_minload_servers(x)` and siblings). SafeTail is free to select any of the 31 non-empty subsets, and converges to $K \approx 3.9$. It therefore spends more compute per request than any policy it is compared against, for 83% of training.

BTP Figure 5.2 does not isolate the contribution of learned scheduling. Some unknown share of the p95/p99 gap is explained by replication budget alone — and redundancy is, by the report's own §6.1, "the most effective known countermeasure against tail latency."

This is the finding most likely to be raised at review, and it is cheap to resolve: re-run the baselines at $K = 4$ and $K = 5$, or subsample SafeTail's decisions to a matched mean K , and report the comparison at equal budget. Until that is done, no claim of the form "SafeTail beats MinLoad/MinProp/Random at the tail" is supportable.

12.5 There are no queues anywhere in the system HIGH

§8.2 and §8.3 of the base report note that the M/M/1 model is absent. Tracing the runtime shows the situation is stronger than "not modelled analytically": **no component buffers requests at all.**

Component	What is actually there	Evidence
Edge server	<code>MAX_CONCURRENT_REQUESTS = 4</code> . When full, <code>schedule_request</code> returns <code>(False, "server full", ...)</code> — the request is <i>rejected, not buffered</i> . This is a c=4 loss system (Erlang-B) , not M/M/1. <code>active_requests</code> is an in-flight list expired by wall-clock <code>finish_time</code> .	<code>servers.py:13, 199–205, 233–249</code>
Controller	No queue object. <code>send_to_server</code> iterates the chunk under <code>with self.lock:</code> . The mutex is the queue. No arrival rate, no service rate, no discipline, no length.	<code>controller.py:1106–1109</code>
Receiver	<code>pending</code> is a deque of accepted <i>TCP connections</i> (<code>MAX_QUEUE = 20</code>), not of requests; connections are served synchronously from the accept loop.	<code>receiver.py:246, 294–298</code>
Queue-length accessor	<code>Controller.get_queue_lengths()</code> is the only such method in the repo and is never called .	<code>controller.py:254</code>

The consequence for the results is specific. `request.queue_waiting_time` is computed as wall-clock elapsed time from the receiver's arrival stamp to just after the agent returns an action (`controller.py:649`). It therefore measures *Python execution time* — CSV lookups, regressor inference, a TensorFlow forward pass, lock contention — not queueing. That number feeds both the waiting-time penalty in the episodic reward and `T` in the deadline-satisfaction function `P(T)` . **Reported deadline satisfaction is partly a function of how fast the machine running the experiment happens to be.**

12.6 The arrival process is not Poisson MEDIUM

Bursts contain `random.randint(2, 4)` chunks of 5 requests, separated by `random.uniform(0.2, 0.8)` second sleeps (`sender_bursts.py:107, 145` ; `constants.py:14–18`). Both are uniform, not exponential. Even if a queue existed, the M/M/1 premise asserted in BTP §3.5 and the README would not hold at the source.

12.7 Replay memory stores the post-outcome state HIGH

§9 finding #3 of the base report notes `next_state == state` . There is a second, independent defect on the same path.

`controller.py:724–729` appends the *live* `Request` object into `step_experiences` . `agent.store` — which is what actually flattens the object into a state vector — is not called until `finalize_episode` (`controller.py:798`), at the end of the episode. By that time the same object has been mutated in place:

- `request.total_processing_delay` has been overwritten with the **realised per-server latencies** (`controller.py:671`), replacing the pre-action estimates written at line 584.
- `request.combination` has been overwritten with the contention string, e.g. `"dps"` (`servers.py:215`).

- `request.queue_waiting_time` has been set (`controller.py:649`).

Because `request_to_state_array` flattens every attribute (§11.1), the state vector written into replay memory **contains the outcome of the action it is paired with**, and is not the state the agent observed when it chose that action. This is distinct from the `_build_features` leak in §9 finding #2 — that one contaminates the latency regressors; this one contaminates the RL training signal directly.

12.8 The workload carries no variation MEDIUM

`main.request_factory` sets `message_size = 1024` and `bandwidth = 20` as literals for all requests; only the type letter (`s / p / d`) is drawn at random (`main.py:86–106`). Transmission delay ignores both fields, sampling `random.choice([18.5, 19.2, 20, 21.5, 22]) / 1000` (`servers.py:93–94`). BTP §3.3's "input and output data sizes that determine transmission delay" and "computational characteristics (FLOPs, input dimensions)" have no representation in the code. Every request of a given type is identical.

12.9 The run processes 15,225 requests, not 500,000 MEDIUM

These are two different quantities and the smaller one binds. `total_no_request = 500000` sizes the pre-generated pool in `SenderBursts.__init__`, but the dispatch loop terminates after `no_of_burst = 1000` bursts (`sender_bursts.py:105`). At `E[burst_size] = 3` chunks $\times$ 5 requests, that is $\approx 15,000$ — matching the 15,225 in the logs exactly. BTP Table 4.2's "500,000 requests / 25,000 episodes" overstates the experiment by roughly 33 $\times$. The shipped logs are not truncated; the configuration simply cannot reach the stated figure.

13. Revised remediation priorities

This supersedes the priority list in §10, reordered for the goal of producing a defensible paper rather than a faithful implementation of the three source documents.

#	Action	Ref	Why this order
1	Re-run baselines at K=4 and K=5, or match SafeTail to a fixed mean K, and regenerate Fig 5.2 at equal replication budget.	§12.4	Cheap — no retraining, the baseline harness already sweeps K. Until it is done the central empirical claim is unsupported, and every downstream comparison inherits the confound.
2	Add a cost term in <code> A </code> to the step reward.	§12.1	Fixes the cause rather than the symptom. Without it the policy will re-converge to near-full replication after any other change.
3	Snapshot the state at action time — deep-copy or serialise into <code>step_experiences</code> — instead of storing the live object.	§12.7	One-line class of fix; without it the DQN trains on leaked outcomes and no convergence result means anything.
4	Introduce a tail-referenced reward term (percentile, CVaR, or a target latency τ).	§10 pri. 3	Unchanged from the base report. The stated objective still never enters the objective function. The SafeTail 1.0 τ mechanism is the cheapest source; the Laplace bound of Tail-Learning (arXiv 2312.16883) is the more novel one.
5	Either implement server queueing, or restate the system as an Erlang-B loss model and drop the M/M/1 claims from README, BTP §3.5 and the abstract.	§12.5	Both are defensible; the current mismatch is not. Note that fixing the claim is nearly free, while implementing queues changes every latency number.
6	Replace wall-clock <code>queue_waiting_time</code> with a simulated quantity.	§12.5	Makes results reproducible across machines. Currently a faster CPU improves reported deadline satisfaction.
7	Retrain or relabel the regressors; drop the leaking <code>total_processing_time</code> feature.	§10 pri. 1	Demoted from first only because items 1–3 invalidate the evaluation regardless of how good the latency predictions are.
8	Add the Oracle baseline; run ≥ 3 seeds per configuration and report error bars.	§10 pri. 2	The <code>_1/_2/_3</code> suffixes in <code>results/baselines/</code> are K variants, not seeds. There are currently no repeated runs anywhere.
9	Correct BTP §4.2 to describe the real state, and §6.2 to describe the real access-rate behaviour.	§11.1, §12.2	Documentation-only, but §4.2 currently makes the paper's own architecture look unnecessary.

Addendum prepared by tracing the runtime path end to end (`main.py` → `sender_bursts.py` → `receiver.py` → `controller.py` → `servers.py` → `agent.py`) and by recomputing all statistics from `results/safetail_training_logs/` . Every line number, formula and figure cited above was verified directly against the repository at the time of writing. Figure A.1 is generated from the shipped logs and is reproducible from them.